

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192365013
Name	K.Dhilip

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

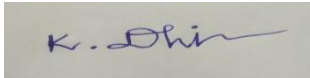
Total Marks: 50

Code of Conduct

I **K.Dhilip/192365013** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

1. **Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.**

For a media platform storing large amounts of videos, images, and audio files, HDFS (Hadoop Distributed File System) can provide reliable, scalable, and cost-efficient storage.

Proposed Design

- **ReplicationFactor=3:**
Each HDFS data block is stored on three different DataNodes. This protects data if one or two nodes fail.
- **HighAvailability:**
Use NameNode High Availability with an Active and Standby NameNode. If the Active NameNode fails, the Standby can take over, reducing downtime.
- **DataNodeCluster:**
Store media files across multiple DataNodes. As the amount of media increases, new DataNodes can be added without redesigning the system.
- **FaultTolerance:**
HDFS automatically detects failed DataNodes and recreates missing replicas on healthy nodes, maintaining data availability.
- **CostEfficiency:**
Use commodity servers instead of expensive high-end storage systems. Storage can be expanded gradually by adding more DataNodes.
- **DataPlacement:**
Place replicas on different nodes and, where possible, different racks. This protects data from both node and rack failures.
- **StorageTiering:**
Frequently accessed media can use faster storage, while older or rarely accessed media can be moved to cheaper storage to reduce costs.
- **Monitoring&Balancing:**
Continuously monitor disk usage, node health, replication status, and cluster performance. HDFS rebalancing can distribute data evenly across nodes.

Constraint Satisfaction

Constraint	Solution
Availability > 99.9%	3 replicas + NameNode HA + automatic recovery
Replication ≤ 3	Replication factor fixed at 3
Cost efficient	Commodity hardware + gradual scaling + storage tiering
Growing media data	Add DataNodes when required
failure protection	Replicas distributed across nodes/racks

The proposed HDFS architecture uses 3-way replication, NameNode High Availability, distributed DataNodes, and cost-effective commodity hardware. It provides high availability above 99.9%, remains within the replication limit of 3, and can scale economically as the media platform grows. Thus, HDFS is a suitable solution for the given constraints.

2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.

A suitable MapReduce workflow can process the log data within the 20-minute batch window by dividing the work into smaller parallel tasks and efficiently using the limited cluster nodes.

Proposed Job Design

Input: Large log files containing timestamp, user ID, IP address, status code, and URL.

Map Phase:

- Input files are divided into input splits.
- Each Mapper processes one split independently.
- The Mapper extracts useful information such as:
 - URL requests
 - Error count
 - User activity
 - Response status
- It produces intermediate key-value pairs, for example:
ErrorCode $\rightarrow$ 1

Shuffle and Sort Phase:

- Hadoop groups the same keys together.
- Related records are sent to the same Reducer.
- This balances the workload across the available nodes.

Reduce Phase:

- Reducers aggregate the grouped data.
- For example:
404 → 1250
means 1250 occurrences of HTTP 404 errors.
- The final results are stored in HDFS.

Task Distribution

Suppose the cluster has 4 worker nodes:

Node	Tasks
Node 1	Mapper 1, Reducer 1
Node 2	Mapper 2, Reducer 2
Node 3	Mapper 3, Reducer 3
Node 4	Mapper 4, Reducer 4

The input is divided into multiple splits and distributed among the available nodes. Several map tasks can run in parallel, followed by parallel reduce tasks.

Meeting the 20-Minute Constraint

- Use an appropriate number of small input splits to maximize parallelism.
- Keep the number of reducers suitable for the limited nodes.
- Use combiners to reduce data transferred during Shuffle.
- Process data locally where possible to reduce network traffic.
- Avoid unnecessary data and reduce the size of intermediate output.
- Monitor task execution and handle slow or failed tasks automatically

The MapReduce job should use parallel Mappers, a Combiner, efficient Shuffle/Sort, and balanced Reducers. This makes the best use of limited cluster nodes and helps complete the log-processing batch within the 20-minute deadline while maintaining efficient resource utilization.

3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.

A suitable YARN scheduler policy should balance fair resource sharing, give higher priority to ETL jobs, and automatically recover when a node fails.

Proposed Scheduler Design

Use the YARN Capacity Scheduler with separate queues for different workloads.

Queue	Priority	Resource Allocation
ETL	Highest	50%
Reporting	Medium	25%
Machine Learning	Normal	25%

1. ETL Priority

- ETL jobs receive the highest priority because they are usually time-sensitive and prepare data for other applications.
- Allocate around 50% of cluster resources to the ETL queue.
- If ETL jobs are not running, unused resources can be temporarily used by other queues.

2. Fairness

- Reporting and ML jobs should receive their minimum guaranteed resources.
- Within each queue, resources are shared fairly among jobs.
- Fair scheduling prevents one large ML job from consuming the entire cluster.
- Resource limits can be applied to individual users or applications.

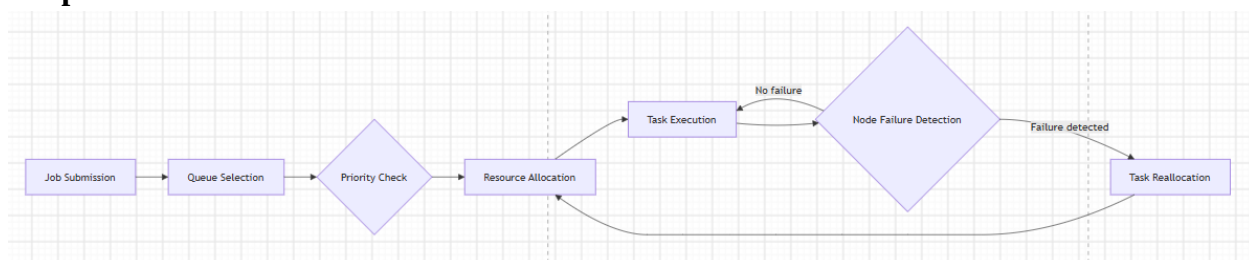
3. Node Failure Recovery

- YARN NodeManager heartbeat monitoring detects failed nodes.
- Containers running on the failed node are marked as failed.
- YARN automatically schedules those tasks again on healthy nodes.
- Application state/checkpoints should be stored in reliable distributed storage so jobs can resume after failure.

4. Dynamic Resource Allocation

The scheduler should allow unused resources to be borrowed by other queues. For example, if ETL is using only 20% of its allocated resources, reporting and ML jobs can temporarily use the remaining capacity.

Simple Workflow

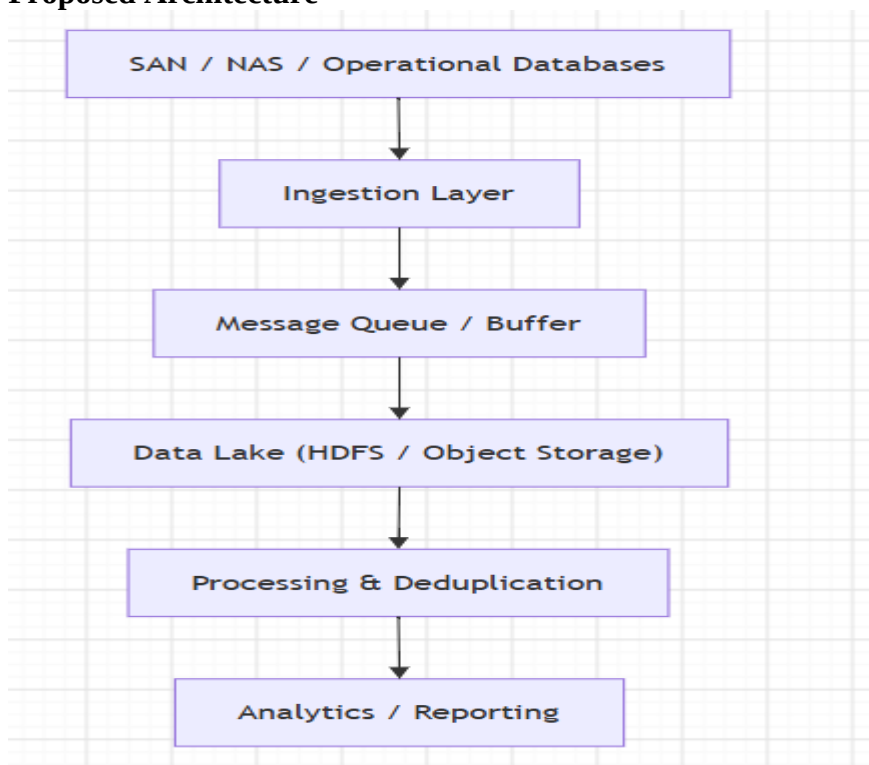


The proposed YARN policy gives highest priority to ETL, maintains fair resource sharing for reporting and ML workloads, and provides automatic recovery from node failures. Dynamic resource allocation also improves cluster utilization and prevents resources from remaining idle.

4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.

An enterprise can use a centralized big data platform with separate ingestion methods for SAN, NAS, and operational databases. The architecture should focus on reliable backup, minimal data duplication, scalability, and efficient storage.

Proposed Architecture



1. SAN Storage

- Use an ingestion gateway to read files from SAN.
- Transfer only new or modified files using incremental synchronization.
- This avoids repeatedly copying the same data.

2. NAS Storage

- Use file-based ingestion tools to collect documents, logs, images, and other files.
- Maintain file metadata such as filename, timestamp, and checksum.
- Checksums help identify duplicate files before storing them.

3. Operational Databases

- Use Change Data Capture (CDC) or incremental queries instead of full database copies.
- Only changed records are transferred.
- This reduces network traffic and duplicate data.

4. Ingestion Layer

Tools such as Apache NiFi or Kafka can manage data movement.

- Provides controlled data flow.
- Handles failures and retries.
- Buffers data during temporary destination failures.

5. Data Lake

Store ingested data in HDFS or cloud object storage.

- Use a common storage format such as Parquet for structured data.
- Organize data by source/date to simplify management.

6. Deduplication

Use checksums/hashes and unique record IDs to identify duplicate data.

- Avoid storing the same file or database record multiple times.
- Maintain metadata/catalog information to track data lineage.

7. Backup and Recovery

- Use HDFS replication or object-storage replication for fault tolerance.
- Maintain separate backup copies for critical data.
- Regularly test restoration to ensure backups are actually recoverable.

Constraint Justification

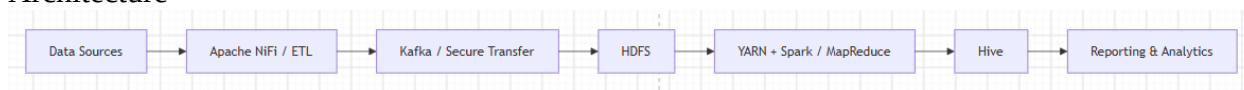
Constraint	Solution
SAN ingestion	Incremental file synchronization
NAS ingestion	Metadata + checksum-based ingestion
Database ingestion	CDC/incremental extraction
Minimal duplication	Hashes, IDs, and deduplication
Backup reliability	Replication + separate backups
Scalability	Distributed data lake

The proposed architecture uses incremental ingestion, CDC, checksums, distributed storage, replication, and reliable backup mechanisms. It minimizes duplicate data while ensuring that important enterprise data can be recovered after failures. This provides a reliable, scalable, and cost-efficient big data ingestion solution.

5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

An enterprise can design a Hadoop ecosystem that securely collects data through Apache NiFi/ETL pipelines, processes it using YARN and MapReduce/Spark, and stores it in HDFS with high availability.

Architecture



1. Secure Data Movement

- Use Apache NiFi to collect data from databases, applications, SAN/NAS, and other sources.
- Use TLS/SSL encryption while transferring data.
- Apply Kerberos authentication to verify users and services.
- Use access controls so only authorized users can access sensitive data.

2. Reliable Data Storage

- Store data in HDFS using a replication factor of 3.
- Replicas are distributed across different DataNodes/racks.
- If one DataNode fails, another replica can serve the data.

3. High Availability

- Configure NameNode High Availability with Active and Standby NameNodes.
- Use ResourceManager HA for YARN.
- Failed containers/tasks are automatically restarted on healthy nodes.
- This prevents a single component failure from stopping the entire system.

4. NiFi / ETL Integration

Apache NiFi manages the complete data flow:

- Extract data from different sources.
- Validate and transform the data.
- Remove duplicate records.
- Route data to HDFS or Kafka.
- Provide monitoring, tracking, and retry mechanisms.

For large batch processing, ETL tools can perform transformations before loading the data into Hadoop.

5. Processing and Analytics

- YARN manages cluster resources.
- Spark/MapReduce processes large datasets.
- Hive provides SQL-based querying and reporting.
- Processed results can be provided to dashboards and business applications.

Constraint Satisfaction

Constraint	Design Solution
Secure data movement	NiFi + TLS/SSL + Kerberos
High availability	NameNode HA + ResourceManager HA
Data reliability	HDFS 3-way replication
ETL integration	Apache NiFi + ETL pipelines
Failure recovery	Automatic task/container restart
Scalability	Distributed Hadoop cluster
Monitoring	NiFi and Hadoop monitoring

This design provides a secure, highly available, and scalable Hadoop ecosystem. Apache NiFi/ETL handles reliable data ingestion, HDFS provides fault-tolerant storage, while YARN, Spark/MapReduce, and Hive provide processing and analytics. Thus, the architecture satisfies the constraints of secure data movement, high availability, and seamless ETL integration.